XML

M. KASSE

Université Alioune DIOP de BAMBEY

youssou.kasse@uadb.edu.sn

D2AW

2014 2015

Chapitre 5

L’extraction de fragments : XPath, XQuery

Introduction

- Comme son nom l'indique, XPath est une spécification fondée sur l'utilisation de chemin d'accès permettant de se déplacer au sein du document XML.
- Dans ce but, un certain nombre de fonctions ont été définies. Elles permettent de traiter les chaînes de caractères, les booléens et les nombres.

Introduction

- XPath est un langage avec une syntaxe non XML, permettant d'adresser les différents noeuds ou groupes de noeuds particuliers d'un document XML.
- XPath voit le document XML comme un arbre de noeuds, qu'il permet de parcourir selon des axes (fils, parent, ancêtre, descendant, ...) et en sélectionnant les noeuds par leur nom.
- Les expressions XPath permettent également de représenter des chaînes de caractères, des nombres et des booléens.

Introduction

- XPath permet d'**effectuer des requêtes** dans un document XML.
- XPath est aux documents XML, ce qu'est SQL aux bases de données.
- XPath est **utilisé par XSLT** pour faire de la transformation de documents XML.

Chemin de localisation

- Un chemin de localisation peut être de type absolu ou relatif.
  - Dans le cas où il est de type **absolu**, il commence toujours par le signe / indiquant la racine du document XML ;
  - Dans le cas où il est de type **relatif**, le noeud de départ est le noeud contextuel courant.

Chemin de localisation

- Un chemin de localisation est composé de **trois parties** :
  - un **axe**, définissant le sens de la relation entre le noeud courant et le jeu de noeuds à localiser;
  - un **noeud** spécifiant le type de noeud à localiser;
  - 0 à n **prédicats** permettant d'affiner la recherche sur le jeu de noeuds à récupérer.

Chemin de localisation

- Par exemple, dans le chemin child::section[position()=1],
  - child est le nom de l'axe,
  - section le type de noeud à localiser (élément ou attribut)
  - [position()=1] est un prédicat.
- Les doubles **::** sont obligatoires.
- La syntaxe d'une localisation s'analyse de gauche à droite.
- Dans notre cas, on cherche dans le noeud courant, un noeud section qui est le premier noeud de son type.

Axes

- **child** : contient les enfants directs du noeud contextuel.

- **descendant** : contient les descendants du nœud contextuel. Un descendant peut être un enfant, un petitenfant...

Axes

- **parent** : contient le parent du noeud contextuel, s'il y en a un.

- **ancestor** : contient les ancêtres du noeud contextuel. Cela comprend son père, le père de son père... Cet axe contient toujours le noeud racine, excepté dans le cas où le nœud contextuel serait lui-même le noeud racine.

Axes

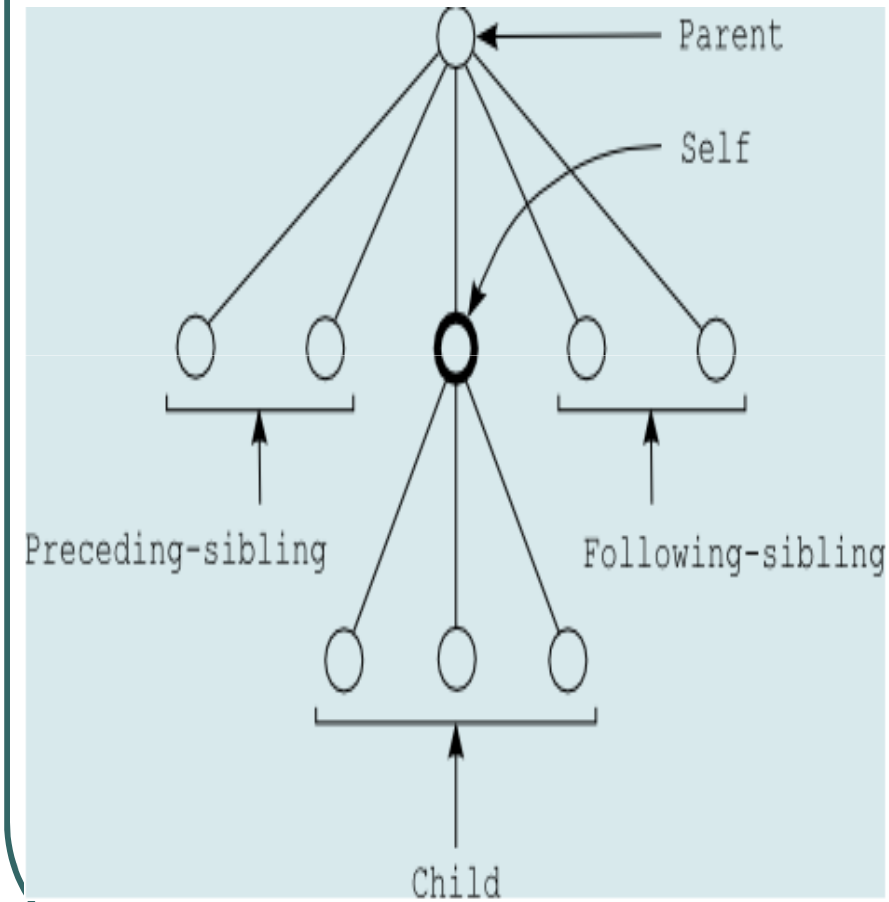
- **following-sibling** : contient tous les noeuds frères qui suivent le noeud contextuel. Dans le cas où ce noeud est un attribut ou un espace de noms, la cible suivante est vide.

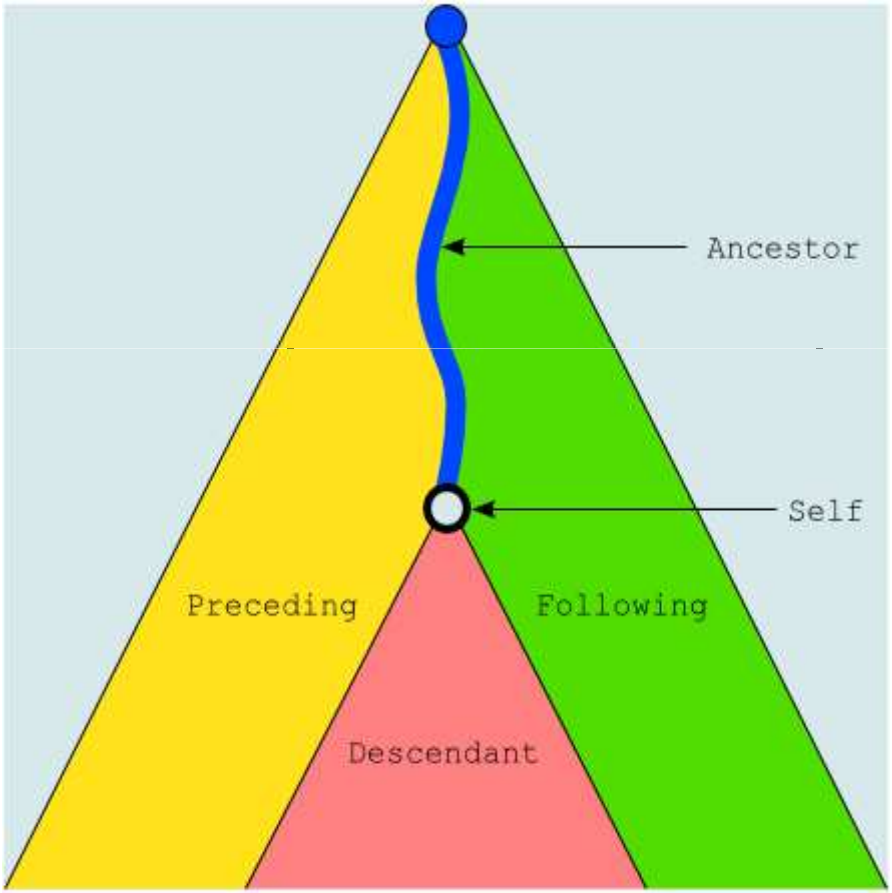
- **preceding-sibling** : contient tous les frères prédécesseurs du noeud contextuel ; si le noeud contextuel est un attribut ou un espace de noms, la cible précédente est vide.

Axes

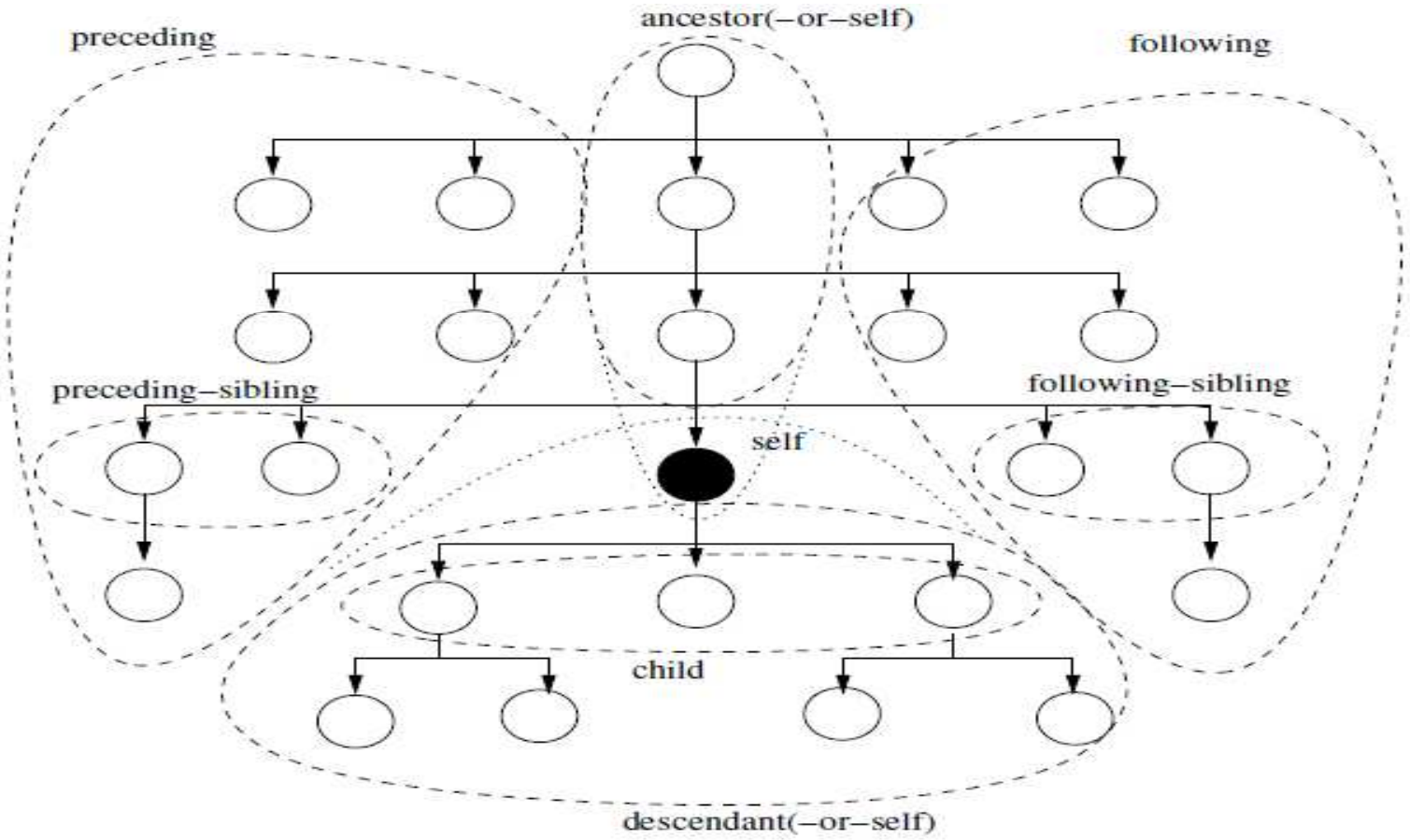
- **following** : contient tous les noeuds du même document que le noeud contextuel qui sont après le noeud contextuel dans l'ordre du document, à l'exclusion de tout descendant, des attributs et des espaces de noms.
- **preceding** : contient tous les prédécesseurs du noeud contextuel ; si le noeud contextuel est un attribut ou un espace de noms, la cible précédente est vide.

Axes Axes


Parent
Self
Preceding-sibling
Following-sibling
Child


Ancestor
Self
Preceding
Following
Descendant

Axes


preceding
ancestor(–or–self)
following
preceding–sibling
following–sibling
self
child
descendant(–or–self)

Axes

- **attribute** : contient les attributs du noeud contextuel ; l'axe est vide quand le noeud n'est pas un élément.
- **namespace** : contient tous les noeuds des espaces de noms du nœud contextuel ; l'axe est vide quand le noeud contextuel n'est pas un élément.
- **self** : contient seulement le noeud contextuel.
- **descendant-or-self** : contient le noeud contextuel et ses descendants.
- **ancestor-or-self** : contient le noeud contextuel et ses ancêtres. Cet axe contiendra toujours le noeud racine

Prédicats

- Un prédicat **mis entre crochets** permet encore de restreindre l'ensemble des nœuds sélectionnés.

- Les prédicats peuvent combiner avec les opérateurs **or** et **and** plusieurs conditions qui peuvent faire intervenir des fonctions prédéfinies.

Fonctions de base

- De nombreuses fonctions peuvent être utilisées. Ces fonctions concernent quatre catégories d'objets : noeuds, chaînes de caractères, booléens, nombres.

- Chaque fonction peut avoir zéro ou plusieurs arguments. Cette liste est loin d'être exhaustive.

Manipulation de nœuds

- *Fonctions retournant un nombre :*
  - last() : retourne un nombre égal à l'index du dernier nœud dans le contexte courant.
  - position() : retourne un nombre égal à la position du nœud dans le contexte courant.
- *Fonction retournant un jeu de nœuds* : id(objet), permet de sélectionner les éléments par leur identifiant.

Manipulation de chaînes de caractères

- Beaucoup de fonctions existent. Citons par exemple
  - **concat(chaine1, chaine2, chaine*)** : concatène toutes les chaînes qui lui sont passées en arguments et retourne la chaîne résultant de cette concaténation.
  - **string-length(chaine?)** : cette fonction retourne le nombre de caractères de la chaîne. Dans le cas où l'argument est omis, la valeur retournée est égale à la longueur de la valeur textuelle du noeud courant.

Exemples

- **child::text ou text**
  - élément text fils d'un nœud du contexte.
- **/child::text ou /text**
  - élément text sous la racine c'est-à-dire contenant tout le document
- **attribute::text ou @text**
  - attribut text d'un nœud du contexte

Exemples

- **child::chapter/child::section** ou **chapter/section**
  - élément section fils d'un nœud chapter lui-même fils d'un nœud du contexte.
- **parent::node()/child::chapter** ou **../chapter**
  - élément chapter frère d'un nœud du contexte.
  -
- **self::node()/descendant-or-self::section** ou **.//section**
  - élément section descendant d'un nœud du contexte.

Exemples

- **child::section[position()=2]** ou **section[2]**
  - deuxième élément section fils du nœud courant.
- **descendant::p/following-sibling::em[position()=1]** ou **//p/following-sibling::em[1]**
  - premier frère em d'un élément p descendant du nœud courant.
- **child::section[child::title]** ou **section[title]**
  - élément section fils du nœud courant et ayant un fils title.

Exemples

- **child::section[attribute::title]** ou **section[@title]**
  - élément section fils du nœud courant et ayant un attribut title.

- **child::section[attribute::title='Un titre']** ou **section[@title='Un titre']**
  - élément section fils du nœud courant et ayant un attribut title égal à la chaîne Un titre.

Exemples

- **chapter[count(child::section) > 1]**
  - élément chapter ayant plus d'un élément section comme fils.
- **@*[name() != 'id']**
  - attributs autres que l'attribut id.
- **section[2][@type='dual']**
  - deuxième élément section du nœud courant si son attribut type vaut dual.

XQuery

- XQuery est un langage de puissant pour extraire des données XML
- Type de données: un seul "document" ou encore des collections sous forme de:
  - fichiers
  - bases de données XML
  - XML "en mémoire" (abres DOM)

XQuery

- Ex: Une requête qui permet très simplement de renvoyer tous les noeuds **auteur** du document biblio.xml.
  - Aucune mise en forme n'étant pratiquée, les nœuds renvoyés le sont avec leurs balises et leur contenu telqu'il s'exprime dans le document.
  - **document("biblio.xml")//auteur**

Les expressions FLWOR

- FLOWR = "For-Let-Where-Order-Return“
-
- rappelle l'idée du select-from-where-..-order-by de SQL

Document Exemple

- <bibliothèque>
  - <livre année="2002">
    - <titre> Prélude à fondation </titre>
    - <auteur><nom> Asimov </nom><prénom> Isaac </prénom></auteur>
    - <éditeur> Pocket </éditeur>
    - <prix>6</prix>
  - </livre>
  - <livre année="1995">
    - <titre> La marche des millénaires </titre>
    - <auteur><nom> Asimov </nom><prénom> Isaac </prénom></auteur>
    - <auteur><nom> White </nom><prénom> Frank </prénom></auteur>
    - <éditeur> Flammarion </éditeur>
    - <prix>8.45</prix>
  - </livre>
  - <livre année="2005">
    - <titre> Avant Dune </titre>
    - <auteur><nom> Anderson </nom>
    - <prénom> Kevin J. </prénom></auteur>
    - <auteur><nom> Herbert </nom><prénom> Brian </prénom></auteur>
    - <éditeur> Pocket </éditeur>
    - <prix>12.3</prix>
  - </livre>
- </bibliothèque>

Les expressions FLWOR

- **for** = itération sur une liste de fragments xml
- **let** = association du résultat d'une expression à une variable
- **where** = condition de sélection
- **order** = tri des résultats
- **return** = expression à retourner

"for"

- **for $variable in expression_recherche RETURN**
- "for" associe à chaque $variable une valeur (fragment XML) trouvé pour l'expression XQuery
- Exemple:
- **for $t in document("biblio.xml")//livre**

let

- "**let**" permet d'assigner une valeur à une variable
-
- requête qui permet d'affecter la variable **$b** successivement avec le contenu des noeuds **livre** du document biblio.xml (clause **for**) et d'affecter à **$a** l'ensemble des nœuds **auteur** contenus dans chaque itération de **$b** (clause **let**).

- **for $b in document("biblio.xml")//livre**
- **let $a := $b/auteur**
- **return <livre nb_auteurs="{count($a)}"> { $a } </livre>**

where

- permet de définir une condition de sélection
- requête pour extraire le **titre** des livres dont le premier noeud fils **auteur** contient comme valeur de **nom** Mansour.
- **for $a in document("biblio.xml")//livre**
- **where $a/auteur[1]/nom eq "Mansour"**
- **return $a/titre**

order

- permet de trier les résultats
- requête pour extraire le **titre** des livres dont le premier noeud fils **auteur** contient comme valeur de **nom** Mansour.
- **for $a in document("biblio.xml")//livre**
- **where $a/auteur[1]/nom eq "Mansour" Order by $a**
- **return $a/titre**

Exemple avec jointure (avec EditiX)

- **Draft de DTD**
- <!ELEMENT base (etudiant | class)*>
- <!ELEMENT etudiant (nom, prenom, adresse)>
- <!ELEMENT class (nom)>
- <!ATTLIST etudiant identifiant ID classe IDREF>
- <!ATTLIST class ident ID>
- **Nom et prénom des étudiant qui sont en WEB2 (nom classe)**

```
<?xml version="1.0" encoding="UTF-8"?>
<liste>
  <etudiant>
    <nom>KARE</nom>
    <prenom>Massamba</prenom>
  </etudiant>
  <etudiant>
    <nom>DIALLO</nom>
    <prenom>Ndiogou</prenom>
  </etudiant>
</liste>
```

<liste>

- for $a in /base/etudiant
- for $b in /base/class
- where ($a/@classe=$b/@ident and $b/nom='WEB2')
- return
- <etudiant>{$a/nom, $a/prenom}
- </etudiant>
- }

</liste>

Expressions conditionnelles

- Structure **if - then - else**
- **if** : teste une condition.
- **then** : opération en cas de réponse positive au test.
- **else** : opération en cas de réponse négative au test.
- Ex : Requête qui va tester pour chaque élément **livre** de biblio.xml si la valeur de son attribut **année** est supérieure à 2000 et renvoyer la valeur 'récent' siggoo c'est la cas et 'ancien' sinon.
- **for $b in document("biblio.xml")//livre**
- **Return**
- **if ($b/@année > 2000)**
- **then 'récent'**
- **else 'ancien'**

XML

M. KASSE

Université Alioune DIOP de BAMBEY

youssou.kasse@uadb.edu.sn

D2AW

2014 2015

Chapitre: 6

eXtensible Stylesheet Langage / Transformation (XSLT)

Introduction

- XSL (eXtended Style Langage) est une recommandation W3C pour spécifier la présentation de documents XML.
- Plus de fonctionnalités que CSS
  - Filtrage de données XML
  - Présentation des données dans n'importe quel ordre
  - Modification et ajout d'information
  - Peut inclure des scripts
- Prise en compte de XML en plus du HTML

Introduction

- Xml permet de séparer le contenu et la présentation
- Avantage de cette séparation
  - Possibilité d'extraire des données du document
  - Possibilité de transformer le contenu
  - Possibilité de publier le document sous différents formats

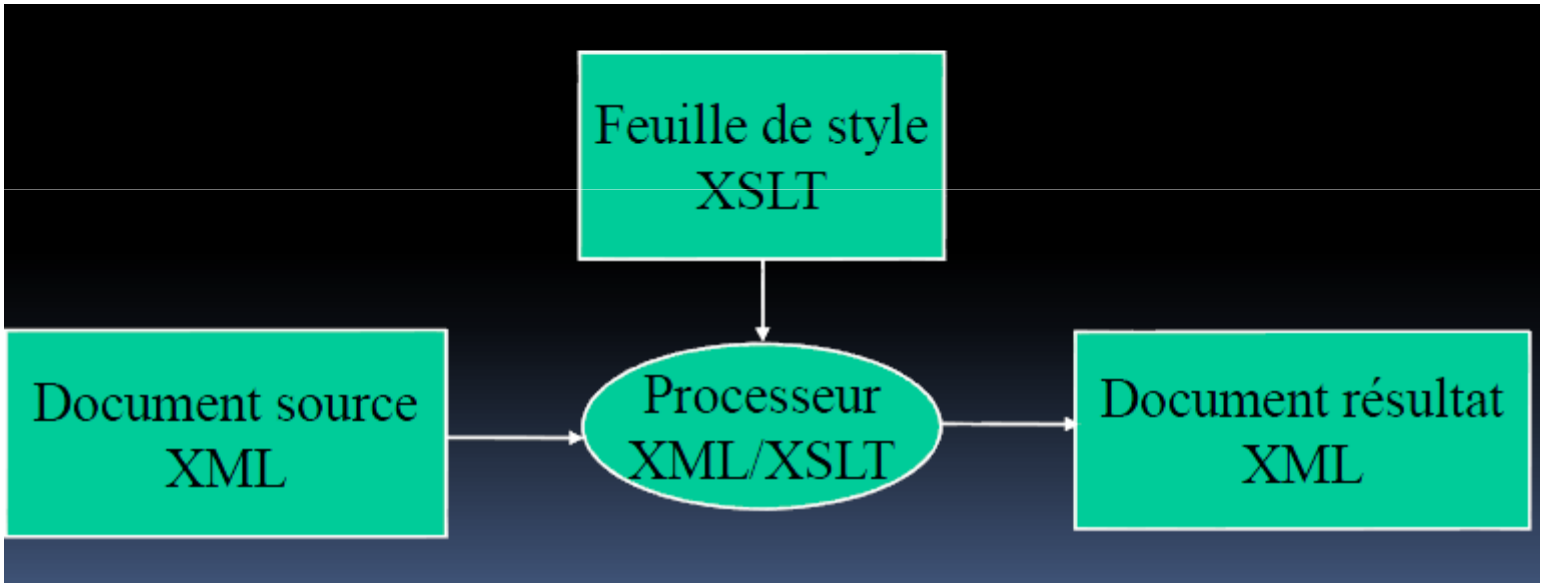
Exemple de sortie

- XML
- <titre>Information</titre>
- HTM
- <h1>Information</h1>
- WML
- <wml><card>Information</card></wml>
- Version Imprimable
- <fo:block>Information</fo:block>

Exemple de sortie

- XSLT permet
  - La transformation des documents XML d'un vocabulaire dans un autre
  - La sélection de contenu spécifique dans un document XML
  - Le tri du contenu d'un élément XML
  - La combinaison du contenu de différents documents XML

Transformation d'un document XML



Principes fondamentaux de XSLT

- Une feuille de style XSL est un document XML
- le même éditeur que pour le document XML
  - Vérifier que la feuille de style est bien formée
  - Pas de DTD pour les feuilles de styles
- XSLT est un langage déclaratif
  - Possède des fonctionnalités très puissantes
  - Différent des langages procéduraux
  - Déclaratif:
    - Un ensemble de template ou modele
    - Composé d'éléments et d'attributs dans l'arbre source
    - Définit l'arbre résultant

Structure d’un document XSL

- Document XSLT
- <xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
- <xsl:transform version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
- L’appel XSL à partir du document XML
- <?xml version="1.0" encoding="ISO-8859-1"?>
- <?xml-stylesheettype="text/xsl" href="cdcatalog.xsl"?>

Structure d'un document

- <?xml version="1.0" encoding="ISO-8859-1"?>
- <xsl:stylesheet version="1.0" xmlns:xsl=http://www.w3.org/1999/XSL/Transform>
- … instructions …
- </ xsl:stylesheet>
- Permet d'informer les parseurs de la présence de feuille style XSL
- Que tous les éléments préfixés de xsl: seront des instructions xsl

Exemple

- <message>Ceci est un message important !</message>
- <xsl:template match="message">
- <H1><xsl:value-of select= "." /></H1>
- </xsl:template>
- Donne ceci:
- <H1>Ceci est un message important !</H1>

Les instructions XSLT

- <xsl:template/>
- C’est la façon la plus simple de lier une règle de traitement et des éléments à traiter
- Le modèle est construit de la façon suivante
  - <xsl:template match="[modèle]">
  - [élément résultant littéraux et éléments XSLT]
  - </xsl:template>
- Modèle peut être un nom d’élément ou une expression identifiant un fragment du document xml

Exemple

- La règle suivante se déclenche dès qu'une balise <projet> est trouvée
- Elle insère dans le document de sortie le contenu de l'élément <title> qui se trouve à l'intérieur d'un sous-élément <problem>

- <xsl:template match="project">
- <P> <xsl:value-of select="problem/title"/> </P>
- </xsl:template>

Exemple

- "Cadre" HTML, puis appel de la règle CINEMA :
- <xsl:template match="/">
  - <html>
    - <head><title>Programme de
    - <xsl:value-of select="CINEMA/NOM"/>
    - </title>
    - </head>
  - <body bgcolor="white">
    - <xsl:apply-templates select="CINEMA"/>
  - </body>
  - </html>
- </xsl:template>

Les instructions XSLT

- <xsl:apply-template/>
  - **<xsl:template match="***expression XPath* **">**
    instructions et/ou sortie à générer
    <xsl:apply-templates/>
    instructions et/ou sortie à générer
    </xsl:template>
  - signifie : "continuer la transformation pour les enfants de l'élément courant (en cours de transformation)

Les instructions XSLT

- **<xsl:template match="/">**
  Liste des tokens :
  </xsl:template>

  <xsl:template match="*taggertoken[@pos != 'PUN']*">
  Forme : **<xsl:value-of select="*@wordform*"/>**
  Lemme : **<xsl:value-of select="*@lemma*"/>**
  </xsl:template>

- Il génère en sortie le texte *"**Liste des tokens**"*, comme spécifié dans le patron. L'arbre a été traité et la transformation s'arrête sans considérer les descendants de la racine. Le deuxième patron n'est jamais utilisé.

Exemple

```
<doc>
  <taggersent>
    <taggertoken wordform="Il" lemma="il" pos="PRO"/>
    <taggertoken wordform="était" lemma="être" pos="VER:sg"/>
    <taggertoken wordform="une" lemma="un" pos="DET:femi:sg"/>
    <taggertoken wordform="fois" lemma="fois" pos="NOM:femi:sg"/>
    <taggertoken wordform="," lemma="," pos="PUN"/>
    <taggertoken wordform="les" lemma="le" pos="DET:masc:pl"/>
    <taggertoken wordform="voleurs" lemma="voleur" pos="NOM:masc:pl"/>
    <taggertoken wordform="étaient" lemma="étayer|être" pos="VER:pl"/>
    <taggertoken wordform="dans" lemma="dans" pos="PRP"/>
    <taggertoken wordform="la" lemma="le" pos="DET:femi:sg"/>
    <taggertoken wordform="forêt" lemma="forêt" pos="NOM:femi:sg"/>
    <taggertoken wordform="." lemma="." pos="PUN"/>
  </taggersent>
</doc>
```

```
<xsl:template match="/">
    Liste des tokens :
</xsl:template>

<xsl:template match="taggertoken[pos != 'PUN']">
    Forme : <xsl:value-of select="@wordform"/>
    Lemme : <xsl:value-of select="@lemma"/>
</xsl:template>
```

Liste des tokens :

Les instructions XSLT

- <<xsl:template match="/">
  Liste des tokens :
  <xsl:apply-templates/>
</xsl:template>

<xsl:template
match="*taggertoken[pos != 'PUN']*">
  Forme : <xsl:value-of
select="@*wordform*"/>
  Lemme : <xsl:value-of
select="@*lemma*"/>
</xsl:template>

- la patron correspondant à la racine est appliqué, le texte *"Liste des tokens"* est reproduit en sortie.
L'instruction **apply-templates** demande au processeur de poursuivre la transformation pour les enfants de l'élément courant.

Exemple

```
<doc>
  <taggersent>
    <taggertoken wordform="Il" lemma="il" pos="PRO"/>
    <taggertoken wordform="était" lemma="être" pos="VER:sg"/>
    <taggertoken wordform="une" lemma="un" pos="DET:femi:sg"/>
    <taggertoken wordform="fois" lemma="fois" pos="NOM:femi:sg"/>
    <taggertoken wordform="," lemma="," pos="PUN"/>
    <taggertoken wordform="les" lemma="le" pos="DET:masc:pl"/>
    <taggertoken wordform="voleurs" lemma="voleur" pos="NOM:masc:pl"/>
    <taggertoken wordform="étaient" lemma="étayer|être" pos="VER:pl"/>
    <taggertoken wordform="dans" lemma="dans" pos="PRP"/>
    <taggertoken wordform="la" lemma="le" pos="DET:femi:sg"/>
    <taggertoken wordform="forêt" lemma="forêt" pos="NOM:femi:sg"/>
    <taggertoken wordform="." lemma="." pos="PUN"/>
  </taggersent>
</doc>
```

```
<xsl:template match="/">
  Liste des tokens :
  <xsl:apply-templates/>
</xsl:template>

<xsl:template match="taggertoken[pos != 'PUN']">
  Forme : <xsl:value-of select="@wordform"/>
  Lemme : <xsl:value-of select="@lemma"/>
</xsl:template>
```

```
Liste des tokens :

Forme : Il
Lemme : il

Forme : était
Lemme : être

Forme : une
Lemme : un

Forme : fois
Lemme : fois

Forme : les
Lemme : le

Forme : voleurs
Lemme : voleur

Forme : étaient
Lemme : étayer|être

Forme : dans
Lemme : dans

Forme : la
Lemme : le

Forme : forêt
Lemme : forêt
```

Utilisation de apply-templates

- Avant d’utiliser des constructions avancées comme "if" ou "for-each", réfléchissez bien si c’est vraiment nécessaire.
- Dans la plupart des cas il suffit de définir des règles avec "xsl:apply-templates" et le reste "s’organise" tout seul.

Les instructions XSLT

- <xsl:for-each select="cle"> instructions </xsl:for-each> :
- répète « instructions » pour chaque élément correspondant au mot-clef "cle" demandé
- Un moyen simple pour remplir des tableaux html avec des contenus d'élément XML

Les instructions XSLT

- <xsl:value-of select =" [expression] "/>
- Permet d'écrire dans le document résultant la valeur d'un élément du document source
- NB: si l'élément du document source possède des éléments enfants, le texte de ces éléments sont aussi inclus
  - xsl:value-of select="TITRE"
  - xsl:value-of select="./@HEURE"

Les instructions XSLT

- <xsl:template match="SALLE">
  - <h2>Salle No
  - <xsl:value-of select="@NO"/></h2>
  - Film: <xsl:value-of select="FILM/TITRE"/>
  - de <xsl:value-of select="FILM/AUTEUR"/>
  -
  - <xsl:for-each select="SEANCES/SEANCE">
    - <xsl:value-of select="."/>
  - </xsl:for-each>
  -
- </xsl:template>

Les instructions XSLT

- Appliqué à Salle1.xml, le résultat (fragment HTML, à intégrer dans un document complet) :
- <h2>Salle No 1 </h2>
  - Film: Alien
  - de: Ridley Scott
-
  - 15:00
  - 18:00
  - 21:00
-

Les instructions XSLT

- <xsl:attribute>
- insère un attribut avec le nom indiqué dans l'arborescence d'un document résultant:
- <xsl:attribute name="nom_attribut">
- Valeur de l'attribut
- </xsl:attribute>

Les instructions XSLT

- <xsl:choose>
- Combiné avec <xsl:when> et <xsl:otherwise>, permet de construire des tests conditionnels à l'instar du switch de Java.
- <xsl:choose>
  - <xsl:when test="condition">
  - instructions
  - ...
  - </xsl:when>
  - ...
  - <xsl:otherwise>
  - instructions
  - ...
  - </xsl:otherwise>
- </xsl:choose>

Les instructions XSLT

- <xsl:if>
- permet d'appliquer un test conditionnel dans la structure d'une feuille de style XSL
- <xsl:if test="condition">
- Instructions...
- </xsl:if>

XSLT avec paramètres

- <html>
  - <head>
  - <title>Formulaire de Recherche</title>
  - </head>
  - <body bgcolor="white">
    - <h1>Formulaire de Recherche</h1>
    - <form method=’get’ action=’Moteur.xml’
    - name=’Form’>
    - Film: <input type=’text’ name=’titre’>
    - Séance: <input type=’text’ NAME=’seance’ >
    - (hh:mm)
    - Ville: <input type=’text’ name=’ville’>
    - <input type=’submit’ name=’chercher’
    - value="Chercher"/>
    - </form>
  - </body>
- </html>

Le document XML

- <?xml version="1.0" encoding="ISO-8859-1"?>
- <!DOCTYPE MOTEUR [
- <!ENTITY EpeeDeBois
- SYSTEM "http://epee-de-bois.fr/EDB.xml">
- <!ENTITY CineMarseille
- SYSTEM "http://cine-marseille.fr/CM.xml">
-]>
- <MOTEUR>
  - <CINEMA>
  - &EpeeDeBois;
  - </CINEMA>
  - <CINEMA>
  - &CineMarseille;
  - </CINEMA>
- </MOTEUR>

Traitement des paramètres

- <xsl:param name="titre"/>
- <xsl:param name="seance"/>
- <xsl:param name="ville"/>
- <xsl:template match="MOTEUR">
  - <xsl:for-each select="CINEMA">
    - <xsl:if test="
      - CINEMA//TITRE = $titre) and
      - CINEMA//HEURE >= $seance) and
      - CINEMA/VILLE = $ville)">
      - <xsl:apply-templates select="." /><p/>
    - </xsl:if>
  - </xsl:for-each>
- </xsl:template>

FIN